

Machine Learning Project 2026/2027 Part I (Regression) -

One-Day-Ahead Chlorophyll-a Forecasting

Alexandre Bernardino, Margarida Silveira, Catarina Barata, Ayman Bnoussaad

September 22, 2026

1 Foreword

The Machine Learning project is composed of one regression and two classification problems. This document presents the regression problem.

At the end of this part, the group must deliver a code package in a specific format described in Section 7.

The code to be delivered will be evaluated automatically. To allow automatic processing, the code must comply with some requirements on the allowed libraries and corresponding versions, as described in Appendix B.

Some functions and objects of Python libraries will not be permitted because they hide important machine learning aspects that students must be exposed to. The set of non permitted functions and objects is described in Section 8.

Students must also comply with fairness rules on group work and use of AI. Templates are provided about these aspects and must be filled and included in the deliverables, as described in Section 10.

Failure to comply with any of the above rules will invalidate the deliverable. To help students verify the rules, validation code is provided to check the code compliance to the rules.

Finally, code will be checked for plagiarism. Detected cases will be penalized according to university rules.

2 Introduction

This first project illustrates the application of linear regression techniques to estimate the parameters of dynamical systems. The objective is one-day-ahead chlorophyll-a forecasting. The prediction of chlorophyll-a is made after the current day's environmental measurements are available but before the current day's chlorophyll measurement is available.

Coastal monitoring programs measure chlorophyll-a because it is widely used as an indicator of phytoplankton biomass. Reliable short-term forecasts can support sampling campaigns, aquaculture operations, water-quality surveillance, and early investigation of harmful algal bloom conditions. The forecasting problem is difficult even at a single monitoring site: ocean variables are seasonal, environmental variables may be statistically correlated, environmental effects may be delayed, and occasional retrieval errors can be much larger than ordinary noise.

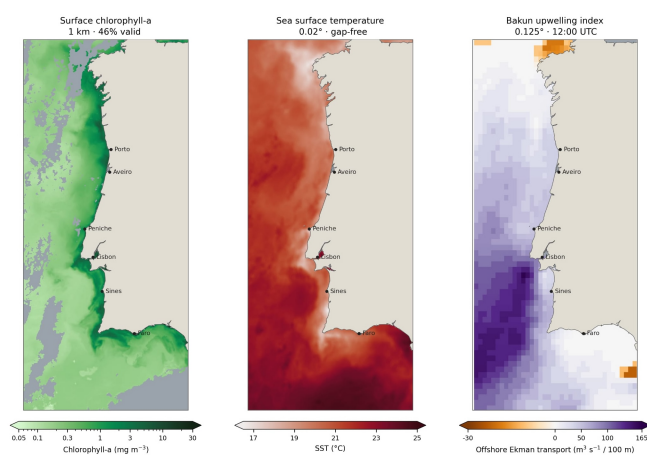


Figure 1: Portuguese Atlantic coast on 09/06/26. Chlorophyll-a concentrations (left) and influencing factors: sea surface temperature (middle) and upwelling index (right).

In this laboratory, you will build a **one-day-ahead predictor of chlorophyll-a concentration** from daily observations at a synthetic coastal station. The dataset is artificial, but it has been designed to reproduce several issues found in real environmental time series. The chlorophyll-a concentration on a given day depends on its own concentration in previous days, and on exogenous variables of that same or past days. As exogenous variables we consider factors that are known to affect the target variable and that can be measured by satellite products and nearby meteorological and environmental monitoring stations, for instance, date, sea surface temperature, nutrients, wind speed, salinity, freshwater river discharge, etc. The full list of dataset variables is provided in Section 4.

3 Problem Formulation

For every day t , predict the chlorophyll-a concentration $y(t)$, in mg/m^3 , using only information available no later than day t . The current day's exogenous measurements may be used; the current day's chlorophyll-a value may not.

A general lagged linear model that approximates the dynamics of chlorophyll-a concentration is:

$$y(t) = c + \sum_{i=1}^p a_i y(t-i) + \sum_{k=1}^K \sum_{j=0}^{q_k} b_{k,j} x_k(t-j) + e(t) \quad (1)$$

where p is the number of lags considered for the target variable, K is the number of exogenous variables, q_k are the lags of each exogenous variable x_k , $y(t-i)$ denote past chlorophyll-a observations, $x_k(t-j)$ denote current or past exogenous measurements, and $e(t)$ denotes measurement noise. This model is of type ARX - Auto-Regressive with eXogenous inputs.

The objective is to use linear regression techniques to estimate the parameters c (an offset, or baseline concentration), a_i (the auto-regressive coefficients), and $b_{k,j}$ (the exogenous variable's coefficients), so we can make predictions at arbitrary dates, assuming the system is stationary (i.e. the parameters do not change with time).

To see how the problem can be formulated as a linear regression, it is more explicit to write the model (1) in regressor form:

$$y(t) = c + \phi(t)^T \theta + e(t) \quad (2)$$

where

$$\phi(t) = [y(t-1), \dots, y(t-p), x_1(t), \dots, x_1(t-q_1), \dots, x_K(t), \dots, x_K(t-q_K)]^T$$

and

$$\theta = [a_1, \dots, a_p, b_{1,0}, \dots, b_{1,q_1}, \dots, b_{K,0}, \dots, b_{K,q_K}]^T$$

This can be written in matrix form:

$$Y = c\mathbf{1} + X\theta + E \quad (3)$$

with $Y = [y(r+1), \dots, y(T)]^T$, $X = [\phi(r+1), \dots, \phi(T)]^T$, $E = [e(r+1), \dots, e(T)]^T$ where T is the size of the dataset (first index is $t = 1$), $\mathbf{1}$ is a column of ones with the same size as Y , and $r = \max(p, q_1, \dots, q_K)$.

You will have to select the candidate lags, p, q_k , $\forall k = 1, \dots, K$, and deal with data pre-processing for preparing the data matrices for linear regression, data centering/normalization, and treatment of unusual observations. For the candidate lags, you can start with reasonable defaults (10-15 days) and then do some exploration around these values.

Intercept convention. In some formulations, the regressor vector begins with a constant component equal to one,

$$\tilde{\phi}(t) = \begin{bmatrix} 1 \\ \phi(t) \end{bmatrix},$$

and the intercept c is included in the corresponding parameter vector. In this laboratory, however, the feature vectors and design matrices produced by the auxiliary code do *not* contain this leading constant component. Accordingly, the intercept is written separately, see Eqs. (2) and (3). When using a regression implementation that estimates an intercept automatically, such as the scikit-learn linear models with `fit_intercept=True`, students should pass the feature matrix X without adding a column of ones. If an estimator is configured with `fit_intercept=False`, the constant component must instead be added explicitly. The two conventions should not be used simultaneously.

4 Dataset and Variables

The complete data set, available only to professors, is composed of 3653 daily samples obtained from 1 January 2015 through 31 December 2024.

The file made available to students, `chlorophyll_student_2015_2023.xlsx` contains only the first 3287 daily samples, obtained from 1 January 2015 through 31 December 2023. The `daily_data` sheet is machine-readable: row 1 contains unique column names, each subsequent row is one day, and each column has one data type.

Appendix A contains some code guidelines on how to read `.xlsx` files and create lagged variables.

Table 1: Variables in the daily chlorophyll-a dataset.

Column	Unit or type	Description
<code>date</code>	Date	UTC observation date; unique and ordered chronologically.
<code>chlorophyll_a_mg_m3</code>	mg m^{-3}	Chlorophyll-a concentration and prediction target.
<code>sst_c</code>	$^{\circ}\text{C}$	Sea-surface temperature.
<code>par_umol_m2_s</code>	$\mu\text{mol m}^{-2} \text{s}^{-1}$	Photosynthetically active radiation.
<code>nitrate_umol_l</code>	$\mu\text{mol L}^{-1}$	Nitrate concentration.
<code>wind_speed_m_s</code>	m s^{-1}	Daily wind speed.
<code>upwelling_index</code>	Dimensionless	Index of coastal upwelling intensity. Larger positive values indicate stronger upwelling.
<code>mixed_layer_depth_m</code>	m	Ocean mixed-layer depth.
<code>salinity_psu</code>	PSU	Practical salinity.
<code>current_speed_m_s</code>	m s^{-1}	Ocean-current speed.
<code>river_discharge_index</code>	Dimensionless	Relative freshwater discharge into the coastal region.
<code>cloud_fraction</code>	0–1	Fraction of the day or observation area covered by clouds.
<code>surface_pressure_hpa</code>	hPa	Atmospheric pressure at the sea surface.
<code>turbidity_ntu</code>	NTU	Water turbidity in nephelometric turbidity units.

Seasonal encoding. Environmental variables and chlorophyll-a concentrations may exhibit an annual seasonal cycle. Using the day of the year directly as a feature would introduce an artificial discontinuity between 31 December and 1 January. To obtain a continuous representation of the annual cycle, the day of the year $d(t)$ can be encoded using two periodic features:

$$s_{\text{year}}(t) = \sin\left(2\pi \frac{d(t)}{365.25}\right), \quad c_{\text{year}}(t) = \cos\left(2\pi \frac{d(t)}{365.25}\right).$$

The value 365.25 approximates the average duration of a year while accounting for leap years. Both sine and cosine are required to identify the position within the annual cycle unambiguously. Dates close to the end and beginning of consecutive years consequently receive similar representations, allowing a linear regression model to capture annual seasonality without treating the calendar-year transition as a large numerical change. These two seasonal features may be included as additional components of the regressor vector $\phi(t)$. Sample code for the computation of the regressor vector $\phi(t)$ is included in the students package file `build_lagged_features.py`.

5 Assessment

Model performance will be evaluated primarily using the coefficient of determination R^2 , which measures predictive fit relative to a constant predictor equal to the mean chlorophyll-a concentration in the test set. Higher values indicate better predictive performance, and R^2 may be negative when a model performs worse than the mean predictor. Students are advised to compute also the Root mean

squared error (RMSE) and mean absolute error (MAE), expressed in mg m^{-3} to quantify prediction errors in physical units.

6 Provided Files

- `ML_LAB_PART_1.pdf`: this handout;
- `data/chlorophyll_student_2015_2023.xlsx` and `.csv`: released data;
- `build_lagged_features.py`: a readable reference implementation of lag creation;
- `declaration_templates/`: editable Word templates for the group declaration of honor and the individual declaration on AI use;
- `student_submission_template/`: required fixed-name submission interface template;
- `validate_submission.py`: local submission-simulation script;
- `model_artifact_policy.py`: policy checks used by the submission simulation to reject not allowed library preprocessing and post-processing objects; and
- `requirements.txt`: supported package versions for the supplied code.

7 Submission format

Submit one ZIP through the Fenix project interface. The system identifies your group from the submission folder name. Do not place the required files inside an extra top-level folder. The ZIP root must contain:

```
best_model/  
  my_model.py  
  model.pkl  
  aux.py  
training_code/  
  train.py  
  <other files used in training>  
declarations/  
  group_declaration_of_honor.pdf  
  student_1_ai_declaration.pdf  
  student_2_ai_declaration.pdf
```

The required `best_model/` directory contains the selected fitted model and all code executed during prediction. The file `my_model.py` defines the grading interface, `model.pkl` stores the fitted predictive model together with plain numerical parameters, and `aux.py` contains any auxiliary prediction functions imported by `my_model.py`. All three filenames are required. Additional runtime files are not allowed outside this directory.

The required `training_code/` directory must contain a main script named `train.py`. It must reproduce the complete training workflow from the released raw data: reading and sorting the data, constructing lagged variables, defining the chronological validation split, calculating every normalization or transformation parameter, selecting and fitting the final model, and saving `best_model/model.pkl`. Other source modules, configuration files, and a short README may be included in this directory. A notebook may be supplied as supplementary material, but it does not replace `train.py`.

The `declarations/` directory must contain the three PDFs shown above. `group_declaration_of_honor.pdf` must use the group template supplied by the teaching team. Each student must complete one individual AI-use declaration, stored as `student_1_ai_declaration.pdf` or `student_2_ai_declaration.pdf`. The order of the students must be the same as in the group declaration. The submission simulation verifies that all three files exist and are valid PDFs.

Do not submit datasets, virtual environments, absolute paths, cached files, or downloaded model binaries. Keep prediction-time files under `best_model/` and training-only files under `training_code/`.

File `best_model/my_model.py` must define exactly the following public entry points:

```
def load_model(model_path):
    """Return the fitted predictor and plain numerical state."""

def predict(model, history_df, day_features_df):
    """Return a finite NumPy array with shape (1,)."""
```

The automated assessment tool loads the model once. For each test date it calls `predict` with:

- **history_df**: A Pandas DataFrame containing all observations strictly before the current prediction date. It has 14 columns: `date`, `chlorophyll_a_mg_m3`, and the 12 environmental predictors. For test-day index $i = 0, \dots, 365$, its dimensions are

$$(3287 + i) \times 14.$$

It therefore has dimensions 3287×14 on 1 January 2024 and 3652×14 on 31 December 2024.

Note that the history used on day $t + 1$ includes the true chlorophyll observation for day t , and not the eventual prediction made at time t .

- **day_features_df**: A Pandas DataFrame with dimensions 1×13 . It contains the current prediction date and the 12 current-day environmental predictors. It does not contain the current-day chlorophyll value.

Your `predict` function must explicitly create its lagged vector, apply the stored numerical normalization parameters, apply any output transformation, preserve input DataFrames, and return one non-negative finite chlorophyll-a concentration. It must not retrain, access the network, read undeclared files, use future rows, or print large outputs.

8 Explicit Code Rules

All preprocessing, feature construction, target transformation, and output post-processing must be written explicitly in `train.py`, `my_model.py`, or your own functions in submitted source files. You may use ordinary NumPy and Pandas operations, for example, to calculate a mean and standard deviation, standardize a matrix, create lagged columns, or other explicit and legible operations. You may also use a library implementation of the **predictive estimator itself**, such as an ordinary least-squares, Ridge, or LASSO estimator.

Do not use a pre-defined library object or convenience function that hides a sequence of preprocessing or post-processing operations. Classes and functions from `sklearn.preprocessing` are permitted, provided that any fitted preprocessing is learned only from the appropriate training data and the same fitted transformation is applied at prediction time. The submission must not use `Pipeline`, `make_pipeline`, `ColumnTransformer`, imputers, feature selectors, feature-extraction or decomposition objects, `TransformedTargetRegressor`, or calibration objects. These restrictions apply during both training and prediction.

The file `model.pkl` may contain the fitted predictive estimator, fitted `sklearn.preprocessing` objects used to transform input features, and plain state such as dictionaries, lists, strings, numbers, and NumPy arrays. A fitted scaler may store its learned means and scales; other required parameters may be saved as plain values. Fit preprocessing objects only on the appropriate training data, and apply those same fitted objects to the corresponding features, in the same column order, during prediction. The file must not contain a `Pipeline`, `ColumnTransformer`, `TransformedTargetRegressor`, or another object prohibited by the submission rules. Lag construction and output post-processing must remain explicit in the submitted source code. The supplied submission simulation and professor-side grader inspect both the Python source and the loaded artifact for these forbidden constructs. This automated check covers ordinary uses; the teaching team may also inspect the source when necessary.

Before zipping, run:

```
python validate_submission.py SUBMISSION_PATH data/chlorophyll_student_2015_2023.xlsx
```

A successful submission simulation checks the fixed `best_model/`, `training_code/`, and `declarations/` structure; the required files; forbidden transformation imports and serialized objects; importability of `my_model.py` and its sibling `aux.py`; callable signatures; model loading; prediction shape; finite values; and accidental mutation of the supplied history. It does not run the training workflow or reveal the hidden test score.

The automated assessment script evaluates all 366 days of 2024 in chronological order. A malformed submission fails early with a short diagnostic. A submission that cannot be imported or violates the interface cannot receive the automated performance component.

9 Reproducibility checklist

Before submitting, confirm that:

- the ZIP has no extra directory level;
- `best_model/` contains `my_model.py`, `model.pkl`, and `aux.py`;
- `training_code/` contains the complete Python training and preprocessing source;
- `training_code/train.py` is the entry point for reproducing the fitted model;
- `declarations/group_declaration_of_honor.pdf` follows the supplied group template;
- `declarations/` contains `student_1_ai_declaration.pdf` and `student_2_ai_declaration.pdf`;
- all required file and directory names match exactly, including case;
- `predict` works after starting Python from a different directory;
- all preprocessing parameters were learned from training data and saved;
- every transformation is implemented explicitly in submitted source code;
- `model.pkl` contains no pipeline, scaler, transformer, or post-processing object;
- lag 0 is never used for chlorophyll-a;
- date sorting is explicit;
- predictions have shape `(1,)`, are finite, and use mg/m3;
- the model does not depend on notebook state, environment variables, or local absolute paths;
- a clean environment can install the listed dependencies; and
- the supplied submission simulation reports `PASS` for the final submission folder.

10 Academic integrity and permitted tools

Follow the course policy on collaboration and external tools. You may use the standard numerical and machine-learning packages available in the declared environment, subject to the submission rules. Library predictive estimators and `sklearn.preprocessing` tools for input features are permitted. Fitted preprocessing must be learned from training data only and applied consistently during prediction. Library pipelines, imputers, feature selectors, target-transformation wrappers, and output post-processing wrappers remain prohibited. Lag construction and any output post-processing must be implemented explicitly in the submitted source code. Your team is responsible for understanding every submitted preprocessing and modelling step. The teaching team may inspect the source code and reproduce the submitted model from `training_code/`.

The `declaration_templates/` directory contains two editable Word templates. Each group must complete and sign one copy of `group_declaration_of_honor_template.docx`. Each student must

separately complete and sign one copy of `individual_ai_use_declaration_template.docx`, including the AI tools used and the purpose and extent of their use. If no AI tool was used, state this explicitly in the individual declaration rather than leaving the corresponding field blank.

Export the completed declarations to PDF and include them in the submitted `declarations/` directory using the exact filenames `group_declaration_of_honor.pdf`, `student_1_ai_declaration.pdf`, and `student_2_ai_declaration.pdf`. The student order must match the order in the group declaration. Do not submit blank templates or editable Word files in place of the required signed PDFs.

Appendix A Working with tabular and lagged data

Excel workbooks in `.xlsx` format can be read into a Pandas `DataFrame`. For the first problem, you can do as follows:

```
import pandas as pd
df = pd.read_excel(
    "chlorophyll_student_2015_2023.xlsx",
    sheet_name="daily_data"
)
```

In this lab you will work with temporal data and sometimes will have to deal with lagged data. To facilitate work with lagged data, data should be organized in chronological order. To make sure dates are interpreted correctly, Excel dates should be converted to Pandas' `datetime` type and sorted in increasing order.

```
df["date"] = pd.to_datetime(df["date"])
df = df.sort_values("date").reset_index(drop=True)
```

Then, lagged variables can be created with the `shift` method. For example, for the first problem:

```
df["chlorophyll_lag_1"] = df["chlorophyll_a_mg_m3"].shift(1)
df["sst_lag_7"] = df["sst_c"].shift(7)
```

Here, the row for day t receives the chlorophyll value from day $t - 1$ and the SST value from day $t - 7$. A lag of zero must never be used for the target variable because it would reveal the value to be predicted.

Rows of `DataFrames` can be joined vertically using:

```
updated_df = pd.concat([df, new_row_df], ignore_index=True)
```

Appendix B Recommended environment and dependencies

To ensure compatibility with the provided code students must use **Python 3.9 through 3.13**; **Python 3.11** is recommended. The project includes a `requirements.txt` file containing the supported dependencies:

```
numpy>=1.24,<3
pandas>=2.0,<3
scipy>=1.10,<2
openpyxl>=3.1,<4
scikit-learn==1.6.1
```

It is advised to use virtual environment with the recommended dependencies:

```
python -m venv .venv
```

On Linux or macOS, activate it with:

```
source .venv/bin/activate
```

On Windows, activate it with:

```
.venv\Scripts\activate
```

The required packages can be installed with:

```
python -m pip install --upgrade pip
python -m pip install --only-binary=:all: -r requirements.txt
```

Students should test their submissions in a clean environment containing only the dependencies listed above. Code that depends on an unlisted package may fail in the automated assessment tools.